

P1280R2

Integer Width Literals

Published Proposal, 2019-06-12

This version:

<https://wg21.link/P1280R1>

Author:

[Isabella Muerte](#)

Audience:

LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1280R1](#)

Current Source:

[slurps-mad-rips/papers/proposals/integer-width-literals.bs](https://ericniebler.com/2019/06/integer-width-literals/)

Table of Contents

1 Revision History

- 1.1 Revision 2
- 1.2 Revision 1
- 1.3 Revision 0

2 Motivation

3 Design

- 3.1 Synopsis

4 Examples

5 Wording

5.1 Feature Testing

References

Informative References

§ 1. Revision History

§ 1.1. Revision 2

- Make all functions `consteval` instead of `constexpr` at the recommendation of Alisdair Meredith.

§ 1.2. Revision 1

- Modify return types to actually be `[u]int_leastXX_t` and friends. This is to make sure that we are actually replacing the `[U]INTxx_C` macros, as these return a `[u]int_leastXX_t`

§ 1.3. Revision 0

Initial Release 🎉

§ 2. Motivation

Proposal [\[p0330r2\]](#) proposes literal suffixes for `ptrdiff_t` and `size_t`. In it, the question [What about the fixed/least/max \(unsigned\) int types?](#) is given regarding the fixed/least/max integer types provided by `<stdint>`. As that paper has decided to focus exclusively on `ptrdiff_t` and `size_t`, this proposal will instead focus on the *fixed* width integer literals. The problem of least/max literals is left for another time and paper. The primary goal of this paper is to replace the `UINTxx_C`/`INTxx_C` macros found in `<stdint>` with user defined literal suffixes.

As an example, one can see the results of the compiler results on [godbolt](#).

```

#include <stdint>

void foo(uint32_t);
void foo(uint64_t);

void test()
{
    // change this to 10ul and it fails to compile on MSVC
    // change this to 10ull and it fails to compile on GCC and Clang
    // The only safe thing to do is to use the UINTXX_C macros.
    auto x = 10;
    foo(x);
}

```

Note: The author is more than willing to extend this paper's scope for the least and max integer types, however the fixed width integer literals show promise of the least complex changes. Also we get to deprecate some C macros in the process and that's always a good thing.

§ 3. Design

The design for these literals suffixes is to permit explicitly sized expressions when writing integer literals (e.g., `123i32 + 12i16`). These operators are declared in the namespace `std::literals::integer_literals` where both `literals` and `integer_literals` are inline namespaces. Access to these operators can be gained with:

- `using namespace std::literals`
- `using namespace integer_literals`
- `using namespace std::literals::integer_literals.`

It is intended that these operators be placed into the `<stdint>` header.

§ 3.1. Synopsis

The operator's full specification is:

```

namespace std::inline literals::inline integer_literals {
    consteval uint_least64_t operator ""u64 (unsigned long long arg);
    consteval uint_least32_t operator ""u32 (unsigned long long arg);
    consteval uint_least16_t operator ""u16 (unsigned long long arg);
    consteval uint_least8_t operator ""u8 (unsigned long long arg);

    consteval int_least64_t operator ""i64 (unsigned long long arg);
    consteval int_least32_t operator ""i32 (unsigned long long arg);
    consteval int_least16_t operator ""i16 (unsigned long long arg);
    consteval int_least8_t operator ""i8 (unsigned long long arg);
}

```

§ 4. Examples

A small code example is presented below to show how the suffix can be used in code. (This code sample assumes that [\[P0645\]](#) has been added to the standard. Additionally as the author of [\[P1276\]](#), we use that here as well)

Note: This code sample does not attempt to solve the previously shown code in [§ 2 Motivation](#), as this is easily solved elsewhere.

```

template <class... Args>
void println (Args&&... args) {
    std::puts(std::format(std::forward<Args>(args)...).c_str());
}

void main () {
    using namespace std::literals;
    println("1 + 2 = {}", 1u32 + 2);
    println("1 - 2 = {}", 1i32 - 2);

    println("0011 AND 0101 is {:04b}", 0b0011u32 & 0b0101);
    println("0011 XOR 0101 is {:04b}", 0b0011u32 ^ 0b0101);
    println("0011 OR 0101 is {:04b}", 0b0011u32 | 0b0101);

    println("One million is written as {}", 1'000'000u32)
}

```

§ 5. Wording

The following is wording for the library section.

```
namespace std::inline literals::inline integer_literals {  
    constexpr uint_least64_t operator ""u64 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<uint_least64_t>::max().

2. Returns

An integer of type uint_least64_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr uint_least32_t operator ""u32 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<uint_least32_t>::max().

2. Returns

An integer of type uint_least32_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr uint_least16_t operator ""u16 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<uint_least16_t>::max().

2. Returns

An integer of type uint_least16_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr uint_least8_t operator ""u8 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<uint_least8_t>::max()

2. Returns

An integer of type uint_least8_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr int_least64_t operator ""i64 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<int_least64_t>::max()

2. Returns

An integer of type int_least64_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr int_least32_t operator ""i32 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= numeric_limits<int_least32_t>::max()

2. Returns

An integer of type int_least32_t with the value given in arg

```
namespace std::inline literals::inline integer_literals {  
    constexpr int_least16_t operator ""i16 (unsigned long long arg);  
}
```

1. Constraints

arg must fit in the range arg <= `numeric_limits<int_least16_t>::max()`.

2. *Returns*

An integer of type `int_least16_t` with the value given in arg.

```
namespace std::inline literals::inline integer_literals {  
    constexpr int_least8_t operator ""i8 (unsigned long long arg);  
}
```

1. *Constraints*

arg must fit in the range arg <= `numeric_limits<int_least8_t>::max()`.

2. *Returns*

An integer of type `int_least8_t` with the value given in arg.

§ 5.1. Feature Testing

The `__cpp_lib_integer_literals` feature test macro should be added.

§ References

§ Informative References

[P0330R2]

JeanHeyd Meneide; Rein Halbersma. [Literal Suffixes for ptrdiff_t and size_t](https://wg21.link/p0330r2). URL: <https://wg21.link/p0330r2>

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645r3). URL: <https://wg21.link/p0645r3>

[P1276]

Isabella Muerte. [Void Main](https://wg21.link/p1276r0). URL: <https://wg21.link/p1276r0>